

1. What are Python data types?

Answer:

Data types tell Python what kind of value is stored in a variable.

Common data types:

- int → Whole numbers (10, 25)
- float → Decimal numbers (10.5, 3.14)
- str → Text ("Hello")
- bool → True or False
- list → Ordered, changeable collection
- tuple → Ordered, not changeable
- set → Unordered, unique values
- dict → Key-value pairs

Example:

```
age = 22      # int
price = 99.5  # float
name = "Bhargavi" # string
is_student = True # boolean
```

2. Difference between List and Tuple

List	Tuple
Uses []	Uses ()
Changeable (Mutable)	Not Changeable (Immutable)
Can add or remove items	Cannot add or remove items
Slightly slower	Faster

Example:

```
my_list = [10, 20, 30]
my_list.append(40)
my_tuple = (10, 20, 30)
# Cannot modify
```

Interview Answer:

A List is mutable, which means we can change it. A Tuple is immutable, which means we cannot change it after creation.

3. Difference between Set and Dictionary

Set	Dictionary
Uses {}	Uses {key:value}
Stores only values	Stores key-value pairs
No duplicate values	Keys must be unique

Example:

```
colors = {"Red", "Blue", "Green"}
```

```
student = {
    "name": "Bhargavi",
    "age": 22
}
```

4. Explain *args and **kwargs

*args

Used to pass **multiple values** to a function.

Example:

```
def add(*numbers):
    print(numbers)
```

```
add(10,20,30)
```

Output

```
(10, 20, 30)
```

**kwargs

Used to pass **multiple keyword arguments**.

```
def student(**details):  
    print(details)
```

```
student(name="Bhargavi", age=22)
```

Output

```
{'name': 'Bhargavi', 'age': 22}
```

Interview Answer:

- *args → Multiple positional arguments
 - **kwargs → Multiple keyword arguments
-

5. What is a Lambda Function?

A **small anonymous function is a function** without a name. It is usually written in a **single line and used** for short operations. It is also called a lambda function in many programming languages.

Syntax

lambda arguments : expression

Example

```
square = lambda x: x*x
```

```
print(square(5))
```

Output

25

Interview Answer

A lambda function is a small one-line function used for simple operations.

6. Difference between == and is ==

Checks whether **values are equal**.

```
a = 10
```

```
b = 10
```

```
print(a == b)
```

Output

True

Is

Checks whether **both variables refer to the same object in memory**.

```
a = [1,2]
```

```
b = a
```

```
print(a is b)
```

Output

True

Interview Answer

- == compares values.
 - is compares memory location (object identity).
-

7. What are Decorators?

Decorators are used to **add extra functionality to a function without changing its original code**.

Example

```
def decorator(func):  
    def wrapper():  
        print("Before function")  
        func()  
        print("After function")  
    return wrapper
```

Interview Answer

A decorator adds extra functionality to a function without modifying the original function.

8. What are Generators?

Generators produce values **one at a time** using the yield keyword.

Example

```
def numbers():  
    yield 1  
    yield 2  
    yield 3
```

```
for i in numbers():  
    print(i)
```

Output

```
1  
2  
3
```

Interview Answer

A generator returns values one by one using yield, which saves memory.

9. Explain Exception Handling

Exception handling is used to **handle errors without stopping the program**.

Example

```
try:  
    print(10/0)  
  
except ZeroDivisionError:  
    print("Cannot divide by zero.")
```

Output

Cannot divide by zero.

Interview Answer

Exception handling uses try and except to handle errors and prevent the program from crashing.

10. What are Modules and Packages?

Module

A module is a **single Python file** containing functions or classes.

Example

```
math.py  
  
import math  
  
print(math.sqrt(25))
```

Package

A package is a **folder containing multiple Python modules**.

Example

```
calculator/  
  add.py  
  sub.py  
  mul.py
```

Interview Answer

- A **Module** is a single Python file.
- A **Package** is a collection of related modules inside one folder.

Easy 1-Line Interview Answers

1. **Python Data Types:** Different types of data like int, float, string, list, tuple, set, and dictionary.
2. **List vs Tuple:** List is mutable; Tuple is immutable.
3. **Set vs Dictionary:** Set stores unique values; Dictionary stores key-value pairs.
4. ***args & **kwargs:** *args accepts multiple positional arguments; **kwargs accepts multiple keyword arguments.
5. **Lambda Function:** A small one-line anonymous function.
6. **== vs is:** == compares values; is compares object identity.
7. **Decorators:** Add extra functionality to a function without changing its code.
8. **Generators:** Return values one at a time using yield.
9. **Exception Handling:** Uses try and except to handle errors.
10. **Modules & Packages:** A module is a Python file; a package is a folder containing multiple modules.

1. Difference between WHERE and HAVING

WHERE

Filters rows

Used before GROUP BY

Cannot use aggregate functions

HAVING

Filters groups

Used after GROUP BY

Can use aggregate functions

Example (WHERE)

```
SELECT * FROM employees  
WHERE salary > 30000;
```

Example (HAVING)

```
SELECT department, COUNT(*)  
FROM employees  
GROUP BY department  
HAVING COUNT(*) > 2;
```

Interview Answer:

WHERE filters individual rows before grouping. HAVING filters grouped data after GROUP BY.

2. Explain INNER JOIN

Definition:

INNER JOIN returns only the matching records from both tables.

Example

```
SELECT e.name, d.department_name  
FROM employees e  
INNER JOIN departments d  
ON e.dept_id = d.dept_id;
```

Interview Answer:

INNER JOIN returns only the rows that have matching values in both tables.

3. What is LEFT JOIN?

Definition:

LEFT JOIN returns all rows from the left table and matching rows from the right table.

If no match is found, NULL is returned.

Example

```
SELECT *  
FROM employees  
LEFT JOIN departments  
ON employees.dept_id = departments.dept_id;
```

Interview Answer:

LEFT JOIN returns all records from the left table and matching records from the right table.

4. What is RIGHT JOIN?

Definition:

RIGHT JOIN returns all rows from the right table and matching rows from the left table.

Example

```
SELECT *  
FROM employees  
RIGHT JOIN departments  
ON employees.dept_id = departments.dept_id;
```

Interview Answer:

RIGHT JOIN returns all records from the right table and matching records from the left table.

5. What is FULL JOIN?

Definition:

FULL JOIN returns all records from both tables.

If there is no match, NULL values are shown.

Example

```
SELECT *  
FROM employees  
FULL OUTER JOIN departments  
ON employees.dept_id = departments.dept_id;
```

Interview Answer:

FULL JOIN returns all rows from both tables, whether they match or not.

6. What is GROUP BY?

Definition:

GROUP BY groups rows with the same values into one group.

Example

```
SELECT department,  
COUNT(*)  
FROM employees  
GROUP BY department;
```

Interview Answer:

GROUP BY is used to group similar rows and perform calculations like COUNT(), SUM(), AVG(), MAX(), and MIN().

7. Difference between COUNT and SUM

COUNT	SUM
Counts rows	Adds numeric values
Works on any column	Works only on numeric columns

COUNT Example

```
SELECT COUNT(*)  
FROM employees;
```

SUM Example

```
SELECT SUM(salary)  
FROM employees;
```

Interview Answer:

COUNT counts records, while SUM adds numeric values.

8. What is a Primary Key?

Definition:

A Primary Key uniquely identifies each row in a table.

Rules

- Cannot contain NULL values.
- Must be unique.

Example

```
CREATE TABLE students(  
id INT PRIMARY KEY,  
name VARCHAR(50)  
);
```

Interview Answer:

A Primary Key uniquely identifies every record in a table.

9. What is a Foreign Key?

Definition:

A Foreign Key connects two tables.

Example

```
CREATE TABLE orders(  
order_id INT,  
customer_id INT,  
FOREIGN KEY(customer_id)  
REFERENCES customers(customer_id)  
);
```

Interview Answer:

A Foreign Key links one table to another using the Primary Key of the parent table.

10. What is Normalization?

Definition:

Normalization organizes data to reduce duplicate data and improve consistency.

Benefits

- Removes duplicate data.
- Saves storage space.
- Improves data consistency.

Interview Answer:

Normalization is the process of organizing data to reduce redundancy and improve data integrity.

11. Write a query to find the second highest salary

Method 1 (Using LIMIT)

```
SELECT DISTINCT salary
FROM employees
ORDER BY salary DESC
LIMIT 1 OFFSET 1;
```

Method 2 (Using MAX)

```
SELECT MAX(salary)
FROM employees
WHERE salary <
(
SELECT MAX(salary)
FROM employees
);
```

12. Delete Duplicate Records

Example

```
DELETE FROM employees
WHERE id NOT IN
(
SELECT MIN(id)
FROM employees
GROUP BY name
);
```

Interview Answer:

Duplicate records can be removed using DELETE with GROUP BY, MIN(), or ROW_NUMBER() (depending on the SQL database).

13. Difference between DELETE, TRUNCATE, and DROP

DELETE	TRUNCATE	DROP
Deletes selected rows	Deletes all rows	Deletes the entire table
Can use WHERE	Cannot use WHERE	Removes table structure too

DELETE

Table remains

TRUNCATE

Table remains

DROP

Table is removed

Examples

```
DELETE FROM employees  
WHERE id=5;
```

```
TRUNCATE TABLE employees;
```

```
DROP TABLE employees;
```

Interview Answer:

- DELETE removes selected rows.
 - TRUNCATE removes all rows but keeps the table.
 - DROP removes the table completely.
-

14. What are Indexes?**Definition:**

Indexes make data searching faster.

Example

```
CREATE INDEX idx_name  
ON employees(name);
```

Interview Answer:

An index improves query performance by allowing SQL to find data faster.

15. What are Transactions?**Definition:**

A transaction is a group of SQL statements that are executed as one unit.

Commands

```
BEGIN;
```

```
UPDATE employees  
SET salary=50000  
WHERE id=1;
```

```
COMMIT;
```

or

ROLLBACK;

Interview Answer:

A transaction ensures that all SQL operations succeed together or fail together. It uses COMMIT to save changes and ROLLBACK to undo changes.

Easy 1-Line Interview Answers

1. **WHERE vs HAVING:** WHERE filters rows; HAVING filters groups.
2. **INNER JOIN:** Returns matching records from both tables.
3. **LEFT JOIN:** Returns all rows from the left table and matching rows from the right table.
4. **RIGHT JOIN:** Returns all rows from the right table and matching rows from the left table.
5. **FULL JOIN:** Returns all rows from both tables.
6. **GROUP BY:** Groups rows with the same values.
7. **COUNT vs SUM:** COUNT counts rows; SUM adds numeric values.
8. **Primary Key:** Uniquely identifies each row.
9. **Foreign Key:** Connects two related tables.
10. **Normalization:** Reduces duplicate data and improves consistency.
11. **Second Highest Salary:** Use ORDER BY ... LIMIT/OFFSET or a subquery with MAX().
12. **Delete Duplicates:** Use DELETE with GROUP BY/MIN() or ROW_NUMBER().
13. **DELETE vs TRUNCATE vs DROP:** DELETE removes rows, TRUNCATE clears all rows, DROP removes the table.
14. **Indexes:** Improve search speed.
15. **Transactions:** Execute multiple SQL statements as one unit using COMMIT and ROLLBACK.

These answers are suitable for **freshers and beginner SQL interviews**, especially for companies like Accenture, TCS, Infosys, Wipro, Capgemini, Cognizant, and HCL.

Git Interview Questions (10 Questions & Answers)

1. What is Git?

Answer:

Git is a **Version Control System (VCS)**.

It helps developers:

- Track changes in code.
- Work with a team.
- Restore previous versions of files.

Example:

If you accidentally delete code, Git helps you recover it.

Interview Answer:

Git is a version control system used to track changes in source code and collaborate with other developers.

2. Difference between Git and GitHub

Git	GitHub
Software installed on your computer	Website that hosts Git repositories
Tracks code changes	Stores code online
Works offline	Requires internet for most features

Interview Answer:

Git is a version control tool. GitHub is a cloud platform that stores Git repositories and helps teams collaborate.

3. What is Git Clone?

git clone copies a remote repository from GitHub to your local computer.

Syntax

git clone https://github.com/user/project.git

Interview Answer:

git clone downloads a GitHub repository to your local machine.

4. What is Git Pull?

git pull downloads the latest changes from the remote repository and updates your local repository.

Syntax

`git pull origin main`

Interview Answer:

git pull fetches the latest changes from GitHub and updates the local repository.

5. What is Git Push?

git push uploads your local commits to the remote repository.

Syntax

`git push origin main`

Interview Answer:

git push sends your local code changes to GitHub.

6. What is a Merge Conflict?

A merge conflict happens when:

- Two developers change the **same part of the same file**.
- Git cannot decide which change to keep.

Example:

- Rahul changes line 10.
- Priya also changes line 10.
- Git asks you to choose or combine the changes.

Interview Answer:

A merge conflict occurs when Git cannot automatically merge changes because the same code was modified in different ways.

7. What is a Branch?

A branch is a separate copy of the code used to develop new features or fix bugs without affecting the main code.

Commands

Create a branch:

`git branch feature`

Switch to a branch:

git checkout feature

Interview Answer:

A branch is an independent line of development used to work on new features or bug fixes.

8. What is a Pull Request (PR)?

A Pull Request is a request to merge your branch into another branch (such as main).

It allows team members to:

- Review your code.
- Suggest changes.
- Approve the merge.

Interview Answer:

A Pull Request is used to request that code from one branch be reviewed and merged into another branch.

9. What is a Commit?

A commit saves your changes in Git with a message describing what you changed.

[Example](#)

git add .

git commit -m "Added login page"

Interview Answer:

A commit is a saved snapshot of your project with a message describing the changes.

10. What is Git Revert?

git revert creates a new commit that **undoes** the changes made by a previous commit.

Syntax

git revert <commit-id>

Interview Answer:

git revert safely undoes a previous commit by creating a new commit.

Common Git Commands

Command	Purpose
git init	Create a new Git repository
git clone	Copy a repository from GitHub
git status	Show changed files
git add .	Stage all changes
git commit -m "message"	Save changes
git push	Upload changes to GitHub
git pull	Download latest changes
git branch	Show or create branches
git checkout branch_name	Switch branches
git merge	Merge one branch into another
git log	Show commit history

Easy 1-Line Interview Revision

1. **Git** → A version control system used to track code changes.
2. **Git vs GitHub** → Git is a tool; GitHub is a cloud platform for Git repositories.
3. **Git Clone** → Downloads a repository to your local computer.
4. **Git Pull** → Gets the latest changes from GitHub.
5. **Git Push** → Uploads local changes to GitHub.
6. **Merge Conflict** → Happens when Git cannot automatically combine changes.
7. **Branch** → A separate line of development for new features or fixes.
8. **Pull Request** → A request to review and merge code into another branch.
9. **Commit** → A saved snapshot of your changes with a message.
10. **Git Revert** → Undoes a previous commit by creating a new commit.

API & Backend Interview Questions (10 Questions & Answers)

1. What is REST API?

Answer

REST API (**Representational State Transfer API**) is a way for two applications to communicate over the internet using HTTP methods like GET, POST, PUT, PATCH, and DELETE.

Example

- You open a food delivery app.
- The app sends a request to the server.
- The server returns the list of restaurants.

This communication happens using a REST API.

Interview Answer:

A REST API is a set of rules that allows applications to communicate with each other using HTTP requests.

2. Difference between GET and POST

GET	POST
Retrieves data	Sends data
Does not change data	Creates new data
Data is sent in the URL	Data is sent in the request body

GET Example

GET /students

Returns all students.

POST Example

POST /students

Creates a new student.

Interview Answer:

GET is used to retrieve data. POST is used to send or create new data.

3. Difference between PUT and PATCH

PUT

Updates the entire resource

Sends all data

PATCH

Updates only specific fields

Sends only changed data

PUT Example

PUT /students/1

Updates the complete student record.

PATCH Example

PATCH /students/1

Updates only one field, such as the student's phone number.

Interview Answer:

PUT updates the entire resource, while PATCH updates only selected fields.

4. What is the DELETE Method?

DELETE is used to remove data from the server.

Example

DELETE /students/5

Deletes the student whose ID is 5.

Interview Answer:

The DELETE method removes a resource from the server.

5. What are HTTP Status Codes?

HTTP status codes tell whether a request was successful or failed.

Code Meaning

200 OK (Success)

201 Created

400 Bad Request

Code Meaning

401 Unauthorized

403 Forbidden

404 Not Found

500 Internal Server Error

Interview Answer:

HTTP status codes indicate the result of a client's request to the server.

6. What is JSON?

JSON (**JavaScript Object Notation**) is a lightweight format used to exchange data between applications.

Example

```
{  
  "id": 101,  
  "name": "Bhargavi",  
  "course": "Python"  
}
```

Interview Answer:

JSON is a text format used to store and exchange data between a client and a server.

7. Explain Request and Response

Request

A request is sent by the **client** (browser or app) to the **server** asking for data or requesting an action.

Response

A response is sent by the **server** back to the **client** with data or a status message.

Example

Client:

GET /students

Server Response:

```
[
  {
    "id": 1,
    "name": "Rahul"
  }
]
```

Interview Answer:

A request is sent from the client to the server. A response is returned by the server to the client.

8. What is Authentication?

Authentication verifies the identity of a user.

Examples:

- Username and password
- OTP
- Fingerprint
- JWT (JSON Web Token)

Interview Answer:

Authentication is the process of verifying that a user is who they claim to be.

9. What is FastAPI?

FastAPI is a modern Python framework used to build APIs.

Features

- Very fast
- Easy to learn
- Automatically generates API documentation
- Supports asynchronous programming

Example

```
from fastapi import FastAPI
```

```
app = FastAPI()
```

```
@app.get("/")
```

```
def home():  
    return {"message": "Hello World"}
```

Interview Answer:

FastAPI is a high-performance Python framework used to create REST APIs quickly and easily.

10. Difference between Flask and FastAPI

Flask	FastAPI
Older framework	Newer framework
Simple and flexible	Fast and modern
Manual API documentation	Automatic API documentation
Slower	Faster

Interview Answer:

Flask is a lightweight web framework, while FastAPI is a modern framework designed for building fast APIs with automatic documentation.

★ **Easy 1-Line Interview Revision**

1. **REST API** → Allows applications to communicate using HTTP.
2. **GET** → Retrieves data.
3. **POST** → Sends or creates new data.
4. **PUT** → Updates the entire resource.
5. **PATCH** → Updates only specific fields.
6. **DELETE** → Removes data from the server.
7. **HTTP Status Codes** → Indicate whether a request succeeded or failed.
8. **JSON** → A format for exchanging data between client and server.
9. **Request & Response** → Client sends a request; server sends a response.
10. **Authentication** → Verifies a user's identity.
11. **FastAPI** → A fast Python framework for building APIs.
12. **Flask vs FastAPI** → Flask is simple and flexible; FastAPI is faster and provides automatic API documentation.

Interview Tip

If you're asked, "How does an API work?", you can answer:

1. The **client** (browser or app) sends an HTTP request.
2. The **server** receives the request.
3. The server processes it (may access a database).
4. The server sends a **response**, usually in **JSON** format.
5. The client displays the result to the user.

Coding problems questions practice

1. Reverse a String

Program

```
text = input("Enter a string: ")  
  
reverse = text[::-1]  
  
print("Reversed String:", reverse)
```

Output

```
Enter a string: hello  
Reversed String: olleh
```

Explanation:

- `[::-1]` reverses the string.

2. Palindrome

A palindrome reads the same forwards and backwards.

Program

```
text = input("Enter a string: ")
```

```
if text == text[::-1]:  
    print("Palindrome")  
else:  
    print("Not Palindrome")
```

Output

Enter a string: madam
Palindrome

3. Fibonacci Series

Program

```
n = int(input("Enter number of terms: "))
```

```
a = 0  
b = 1
```

```
for i in range(n):  
    print(a, end=" ")  
    c = a + b  
    a = b  
    b = c
```

Output

Enter number of terms: 7
0 1 1 2 3 5 8

4. Prime Number

A prime number has only **2 factors: 1 and itself**.

Program

```
num = int(input("Enter a number: "))
```

```
if num > 1:  
    for i in range(2, num):  
        if num % i == 0:  
            print("Not Prime")  
            break  
else:
```



```
    print("Prime")
else:
    print("Not Prime")
```

Output

Enter a number: 13
Prime

5. Armstrong Number

Example:

153

$$1^3 + 5^3 + 3^3 = 153$$

Program

```
num = int(input("Enter a number: "))
```

```
temp = num
sum = 0
```

```
while temp > 0:
    digit = temp % 10
    sum += digit ** 3
    temp //= 10
```

```
if num == sum:
    print("Armstrong Number")
else:
    print("Not Armstrong")
```

Output

Enter a number: 153
Armstrong Number

6. Factorial

Example

$$5! = 5 \times 4 \times 3 \times 2 \times 1 = 120$$

Program

```
num = int(input("Enter a number: "))
```

```
fact = 1
```

```
for i in range(1, num + 1):  
    fact *= i
```

```
print("Factorial:", fact)
```

Output

Enter a number: 5

Factorial: 120

7. Count Vowels

Program

```
text = input("Enter a string: ")
```

```
count = 0
```

```
for ch in text.lower():  
    if ch in "aeiou":  
        count += 1
```

```
print("Vowels:", count)
```

Output

Enter a string: education

Vowels: 5

8. Remove Duplicates from List

Program

```
numbers = [1,2,2,3,4,4,5]
```

```
unique = list(set(numbers))
```

```
print(unique)
```

Output

[1, 2, 3, 4, 5]

9. Find Maximum Number

Program

```
numbers = [10,25,8,40,15]
```

```
print(max(numbers))
```

Output

40

Without using max():

```
numbers = [10,25,8,40,15]
```

```
largest = numbers[0]
```

```
for i in numbers:
```

```
    if i > largest:
```

```
        largest = i
```

```
print(largest)
```

10. Two Sum

Find two numbers whose sum equals the target.

Program

```
nums = [2,7,11,15]
```

```
target = 9
```

```
for i in range(len(nums)):
```

```
    for j in range(i+1, len(nums)):
```

```
        if nums[i] + nums[j] == target:
```

```
            print(i, j)
```

Output

0 1

11. Sorting

Ascending Order

```
numbers = [5,3,8,1]
```

```
numbers.sort()
```

```
print(numbers)
```

Output

```
[1,3,5,8]
```

Descending Order

```
numbers.sort(reverse=True)
```

```
print(numbers)
```

Output

```
[8,5,3,1]
```

12. Binary Search

```
numbers = [10,20,30,40,50]
```

```
target = 30
```

```
low = 0
```

```
high = len(numbers)-1
```

```
while low <= high:
```

```
    mid = (low + high)//2
```

```
    if numbers[mid] == target:
```

```
        print("Found")
```

```
        break
```

```
    elif numbers[mid] < target:
```

```
        low = mid + 1
```

```
    else:
```

```
        high = mid - 1
```

Output

```
Found
```

13. Dictionary-Based Problem (Count Characters)

Program

```
text = "apple"

count = {}

for ch in text:
    count[ch] = count.get(ch,0) + 1

print(count)
```

Output

```
{'a':1,'p':2,'l':1,'e':1}
```

★ Easy 1-Line Interview Revision

Question	Easy Answer
Reverse String	Use slicing[::-1]
Palindrome	Compare string with its reverse
Fibonacci	Each number is the sum of the previous two
Prime Number	A number with only two factors: 1 and itself
Armstrong Number	Sum of digits raised to the power of the number of digits equals the original number
Factorial	Product of all positive integers up to the number
Count Vowels	Count a, e, i, o, u in a string
Remove Duplicates	Use set()
Find Maximum	Use max() or compare elements in a loop
Two Sum	Find two numbers whose sum equals the target
Sorting	Use sort() or sorted()
Binary Search	Search a sorted list by repeatedly checking the middle element
Dictionary Problem	Store data as key: value pairs

CS fundamentals

1. DBMS (Database Management System)

A **DBMS** is software used to **store, manage, and retrieve data**.

Example:

A college stores student details in a database.

ACID Properties

ACID makes sure database data is **safe and correct**.

A - Atomicity

- Either **all work is completed** or **nothing happens**.
- No half-completed transaction.

Example:

You transfer ₹1000 from Account A to B.

- Money should be deducted from A **and** added to B.
 - If one step fails, everything is cancelled.
-

C - Consistency

- Data should always stay **correct**.

Example:

A student's marks cannot become negative.

I - Isolation

- **Multiple users can work at the same time without affecting each other.**

Example:

Two people book train tickets at the same time.
Each transaction works separately.

D - Durability

- Once data is saved, it stays saved even if power goes off.

Normalization

Normalization means **organizing data** to remove duplicate (repeated) data.

Advantages

- Saves storage
 - No duplicate data
 - Easy to update
-

SQL vs NoSQL

SQL	NoSQL
Tables	Documents/Collections
Fixed structure	Flexible structure
MySQL	MongoDB
Best for banking	Best for social media

2. Operating System (OS)

An Operating System manages the computer.

Examples:

- Windows
 - Linux
 - Android
-

Process vs Thread

Process

A process is a **running program**.

Example:

You open Chrome.

Chrome is one process.

Thread

A thread is a **small task inside a process**.

[Example:](#)

Chrome:

- One thread loads YouTube.
- Another thread downloads a file.

Many threads can work inside one process.

Process	Thread
Big task	Small task
Uses more memory	Uses less memory
Independent	Part of process

Deadlock

Deadlock happens when two processes are **waiting for each other forever**.

[Example](#)

Person A has Pen.

Person B has Notebook.

- A waits for Notebook.
- B waits for Pen.

Both wait forever.

This is called **Deadlock**.

Multithreading

Multithreading means **running many threads at the same time**.

[Example](#)

While watching YouTube:

- Video plays

- Comments load
- Ads load

All happen together.

3. Computer Networks

A computer network connects computers so they can communicate.

HTTP vs HTTPS

HTTP

- Not secure
- Data is not encrypted

Example:

<http://example.com>

HTTPS

- Secure
- Data is encrypted

Example:

<https://google.com>

HTTP	HTTPS
Not secure	Secure
No encryption	Encryption

TCP vs UDP

TCP

- Reliable
- Data arrives correctly
- Slower

Used in:

- Banking
 - Email
 - File download
-

UDP

- Faster
- Less reliable
- Some data may be lost

Used in:

- Online games
- Video calls
- Live streaming

TCP

UDP

Reliable

Fast

Slower

Faster

No data loss Data loss possible

4.DNS (Domain Name System)

DNS converts a **website name into an IP address**.

Example

You type:

www.google.com

DNS finds Google's IP address and connects you to the website.

Think of DNS like your phone contacts:

- You search "**Mom**"
- Phone dials her number.

Similarly,

- You type **google.com**
 - DNS finds the IP address.
-

Client-Server Architecture

Client

The device that **requests data**.

Examples:

- Laptop
 - Mobile
 - Browser
-

Server

The computer that **stores data and sends it** to the client.

Example

You open YouTube.

1. Your phone sends a request (Client).
2. YouTube Server receives it.
3. Server sends the video.
4. You watch the video.

Client -----Request-----> Server

Client <-----Response----- Server

★ Interview One-Line Answers

- **DBMS:** Software to store and manage data.
- **ACID:** Ensures safe and reliable database transactions.
- **Normalization:** Removes duplicate data.
- **SQL:** Table-based database.
- **NoSQL:** Flexible document-based database.

- **Process:** A running program.
- **Thread:** A small task inside a process.
- **Deadlock:** Two processes waiting for each other forever.
- **Multithreading:** Running multiple threads simultaneously.
- **HTTP:** Not secure.
- **HTTPS:** Secure version of HTTP.
- **TCP:** Reliable communication protocol.
- **UDP:** Fast communication protocol.
- **DNS:** Converts website names into IP addresses.
- **Client:** Requests data.
- **Server:** Provides data.

1. What is Object-Oriented Programming (OOP)?

Answer:

OOP is a programming method where we create **objects** and **classes**.

It helps us organize code, reuse code, and make programs easier to understand.

Example:

- Car is a class.
- BMW, Audi, Tesla are objects.

2. Explain the Four Pillars of OOP

The four pillars are:

1. Encapsulation

Keeping data and methods together in one class.

2. Abstraction

Showing only important details and hiding unnecessary details.

3. Inheritance

One class can use the properties and methods of another class.

4. Polymorphism

One method can work in different ways.

3. Difference Between Abstraction and Encapsulation

Abstraction	Encapsulation
Hides implementation	Hides data
Shows only important features	Protects data
Focus on "What?"	Focus on "How?"

Simple Example

ATM Machine

- **Abstraction:** You only press buttons.
- **Encapsulation:** The ATM hides your account details.

4. What is Inheritance?

Inheritance means **one class gets properties and methods from another class.**

It helps reuse code.

```
class Animal:  
    def sound(self):  
        print("Animal Sound")
```

```
class Dog(Animal):  
    pass
```

```
d = Dog()  
d.sound()
```

Output

Animal Sound

5. Types of Inheritance in Python

There are **5 types**.

1. Single Inheritance
2. Multiple Inheritance
3. Multilevel Inheritance

4. Hierarchical Inheritance
 5. Hybrid Inheritance
-

6. Method Overloading vs Method Overriding

Method Overloading

Same method name with different parameters.

Python does **not support true method overloading**.

Method Overriding

Child class changes the parent's method.

Example

```
class Animal:
    def sound(self):
        print("Animal")

class Dog(Animal):
    def sound(self):
        print("Bark")
```

7. What is Constructor (__init__)?

A constructor is a special method that runs **automatically when an object is created**.

Example

```
class Student:
    def __init__(self, name):
        self.name = name

s = Student("Bhargavi")
print(s.name)
```

Output

Bhargavi

8. What is self?

self refers to the **current object** of the class.

It is used to access variables and methods inside the class.

Example

```
class Student:
    def __init__(self, name):
        self.name = name
```

Here,

`self.name`

belongs to that object.

9. Static Method vs Class Method

Static Method

- Uses `@staticmethod`
- Does not use `self` or `cls`.

Class Method

- Uses `@classmethod`
- Uses `cls`.
- Works with the class.

Example

class Demo:

```
@staticmethod
def hello():
    print("Hello")
```

```
@classmethod
def welcome(cls):
    print("Welcome")
```

10. What is Polymorphism?

Polymorphism means **one method behaves differently in different classes**.

Example

```
class Dog:
    def sound(self):
        print("Bark")
```

```
class Cat:
    def sound(self):
        print("Meow")
```

Both have a sound() method but produce different outputs.

11. What is Composition?

Composition means **one class contains another class**.

Example

```
class Engine:
    pass
```

```
class Car:
    def __init__(self):
        self.engine = Engine()
```

A Car has an Engine.

12. What is super()?

super() is used to call the **parent class method or constructor**.

Example

```
class Parent:
    def __init__(self):
        print("Parent")
```

```
class Child(Parent):
    def __init__(self):
        super().__init__()
        print("Child")
```

Output

Parent
Child

13. Can Python Support Multiple Inheritance?

Yes.

Python supports multiple inheritance.

Example

```
class Father:  
    pass
```

```
class Mother:  
    pass
```

```
class Child(Father, Mother):  
    pass
```

The child class inherits from both parents.

14. What is an Abstract Class?

An abstract class is a class that **cannot be used to create objects directly**.

It contains one or more **abstract methods** that child classes must implement.

Example

```
from abc import ABC, abstractmethod
```

```
class Animal(ABC):
```

```
    @abstractmethod  
    def sound(self):  
        pass
```

15. Explain a Real-Time OOP Example

Example: Car

Class: Car

Objects: BMW, Audi, Tesla

Features (Attributes):

- Color
- Model

- Speed

Actions (Methods):

- Start()
- Stop()
- Drive()

Example

class Car:

```
def start(self):  
    print("Car Started")
```

```
car = Car()  
car.start()
```

Output

Car Started

★ **Short Interview Answers (1–2 lines)**

1. **OOP:** A programming method that uses classes and objects.
2. **Four Pillars:** Encapsulation, Abstraction, Inheritance, Polymorphism.
3. **Abstraction vs Encapsulation:** Abstraction hides implementation; Encapsulation hides data.
4. **Inheritance:** Child class uses parent class properties and methods.
5. **Types of Inheritance:** Single, Multiple, Multilevel, Hierarchical, Hybrid.
6. **Overloading vs Overriding:** Overloading = same method name with different parameters (not fully supported in Python); Overriding = child class changes parent method.
7. **Constructor:** `__init__()` runs automatically when an object is created.
8. **self:** Refers to the current object.
9. **Static vs Class Method:** Static uses `@staticmethod`; Class method uses `@classmethod` and `cls`.
10. **Polymorphism:** Same method name behaves differently in different classes.
11. **Composition:** One class contains another class.
12. **super():** Calls the parent class constructor or methods.
13. **Multiple Inheritance:** Yes, Python supports it.

14. **Abstract Class:** A class that cannot be instantiated and contains abstract methods.

15. **Real-time Example:** A Car class with objects like BMW, Audi, and Tesla.

HR Interview Tips

1. Tell me about yourself.

Answer:

"Good morning, sir/madam.

My name is **B. Bhargavi**. I completed my **Bachelor's degree** in **[Your Branch]** from **[Your College Name]**.

I have learned **Python, SQL, Power BI, MongoDB, and basic DBMS, Operating System, and Computer Networks**.

I have done academic projects to improve my technical skills.

I am a quick learner, hardworking, and always willing to learn new technologies.

I am looking for an opportunity to start my career in a company like Accenture where I can learn and grow.

Thank you."

2. Why Accenture?

Answer:

"I want to join Accenture because it is one of the world's leading IT companies.

Accenture provides a good learning environment, training programs, and career growth opportunities.

I believe I can improve my technical and professional skills while contributing to the company's success."

3. Why Python?

Answer:

"I like Python because it is easy to learn and understand.

Python has simple syntax and is used in web development, data analysis, artificial intelligence, and automation.

As a beginner, Python helped me understand programming easily."

4. Describe a challenging situation.

Answer:

"During my college project, our team had very little time to complete the project.

We divided the work among team members and helped each other.

We completed the project on time and learned the importance of teamwork and time management."

5. Have you worked in a team?

Answer:

"Yes.

During my college project, I worked with my teammates.

We shared responsibilities, discussed ideas, solved problems together, and completed the project successfully."

6. Are you willing to relocate?

Answer:

"Yes.

I am willing to relocate wherever the company requires me."

7. How do you handle deadlines?

Answer:

"I first understand the task.

Then I make a plan, set priorities, and complete important work first.

If I face any problem, I ask for help instead of delaying the work.

This helps me finish work on time."

8. Where do you see yourself in 5 years?

Answer:

"In five years, I see myself as a skilled software professional.

I want to improve my technical knowledge, take more responsibilities, and contribute to the company's success."

9. Why should we hire you?

Answer:

"I am a quick learner, hardworking, and eager to learn new technologies.

I have basic knowledge of Python, SQL, Power BI, and database concepts.

I have a positive attitude, good communication skills, and I am ready to work hard for the company."

10. Do you have any questions for us?

Answer:

You can ask:

- "What technologies will I be working on?"
 - "What training programs are provided for freshers?"
 - "How do you measure employee performance?"
-

★ **Bonus: HR Interview Tips**

- 😊 Smile and be confident.
- 👔 Dress professionally.
- 👁️ Maintain eye contact.
- 🗣️ Speak slowly and clearly.
- ✖️ Don't say "I don't know" immediately. Try to answer calmly.
- ⌚ Keep each answer between **30–60 seconds**.

Opps concept (15 questions)

1. What is Object-Oriented Programming (OOP)?

Answer:

OOP is a programming method where we create **objects** and **classes**.

It helps us organize code, reuse code, and make programs easier to understand.

Example:

- Car is a class.

- BMW, Audi, Tesla are objects.

2. Explain the Four Pillars of OOP

The four pillars are:

1. Encapsulation

Keeping data and methods together in one class.

2. Abstraction

Showing only important details and hiding unnecessary details.

3. Inheritance

One class can use the properties and methods of another class.

4. Polymorphism

One method can work in different ways.

3. Difference Between Abstraction and Encapsulation

Abstraction	Encapsulation
Hides implementation	Hides data
Shows only important features	Protects data
Focus on "What?"	Focus on "How?"

Simple Example

ATM Machine

- **Abstraction:** You only press buttons.
- **Encapsulation:** The ATM hides your account details.

4. What is Inheritance?

Inheritance means **one class gets properties and methods from another class.**

It helps reuse code.

```
class Animal:  
    def sound(self):
```

```
print("Animal Sound")
```

```
class Dog(Animal):  
    pass
```

```
d = Dog()  
d.sound()
```

Output

Animal Sound

5. Types of Inheritance in Python

There are **5 types**.

1. Single Inheritance
 2. Multiple Inheritance
 3. Multilevel Inheritance
 4. Hierarchical Inheritance
 5. Hybrid Inheritance
-

6. Method Overloading vs Method Overriding

Method Overloading

Same method name with different parameters.

Python does **not support true method overloading**.

Method Overriding

Child class changes the parent's method.

Example

```
class Animal:  
    def sound(self):  
        print("Animal")
```

```
class Dog(Animal):  
    def sound(self):  
        print("Bark")
```

7. What is Constructor (__init__)?

A constructor is a special method that runs **automatically when an object is created**.

Example

```
class Student:
    def __init__(self, name):
        self.name = name

s = Student("Bhargavi")
print(s.name)
```

Output

Bhargavi

8. What is self?

self refers to the **current object** of the class.

It is used to access variables and methods inside the class.

Example

```
class Student:
    def __init__(self, name):
        self.name = name
```

Here,

self.name

belongs to that object.

9. Static Method vs Class Method

Static Method

- Uses @staticmethod
- Does not use self or cls.

Class Method

- Uses @classmethod
- Uses cls.
- Works with the class.

Example

```
class Demo:
```

```
    @staticmethod
    def hello():
        print("Hello")
```

```
    @classmethod
    def welcome(cls):
        print("Welcome")
```

10. What is Polymorphism?

Polymorphism means **one method behaves differently in different classes**.

Example

```
class Dog:
    def sound(self):
        print("Bark")
```

```
class Cat:
    def sound(self):
        print("Meow")
```

Both have a sound() method but produce different outputs.

11. What is Composition?

Composition means **one class contains another class**.

Example

```
class Engine:
    pass
```

```
class Car:
    def __init__(self):
        self.engine = Engine()
```

A Car has an Engine.

12. What is super()?

super() is used to call the **parent class method or constructor**.

Example

```
class Parent:
    def __init__(self):
        print("Parent")

class Child(Parent):
    def __init__(self):
        super().__init__()
        print("Child")
```

Output

Parent
Child

13. Can Python Support Multiple Inheritance?

Yes.

Python supports multiple inheritance.

Example

```
class Father:
    pass

class Mother:
    pass

class Child(Father, Mother):
    pass
```

The child class inherits from both parents.

14. What is an Abstract Class?

An abstract class is a class that **cannot be used to create objects directly**.

It contains one or more **abstract methods** that child classes must implement.

Example

```
from abc import ABC, abstractmethod

class Animal(ABC):
```

```
@abstractmethod
def sound(self):
    pass
```

15. Explain a Real-Time OOP Example

Example: Car

Class: Car

Objects: BMW, Audi, Tesla

Features (Attributes):

- Color
- Model
- Speed

Actions (Methods):

- Start()
- Stop()
- Drive()

Example

```
class Car:
```

```
    def start(self):
        print("Car Started")
```

```
car = Car()
car.start()
```

Output

Car Started

★ Short Interview Answers (1–2 lines)

1. **OOP:** A programming method that uses classes and objects.
2. **Four Pillars:** Encapsulation, Abstraction, Inheritance, Polymorphism.

3. **Abstraction vs Encapsulation:** Abstraction hides implementation; Encapsulation hides data.
4. **Inheritance:** Child class uses parent class properties and methods.
5. **Types of Inheritance:** Single, Multiple, Multilevel, Hierarchical, Hybrid.
6. **Overloading vs Overriding:** Overloading = same method name with different parameters (not fully supported in Python); Overriding = child class changes parent method.
7. **Constructor:** `__init__()` runs automatically when an object is created.
8. **self:** Refers to the current object.
9. **Static vs Class Method:** Static uses `@staticmethod`; Class method uses `@classmethod` and `cls`.
10. **Polymorphism:** Same method name behaves differently in different classes.
11. **Composition:** One class contains another class.
12. **super():** Calls the parent class constructor or methods.
13. **Multiple Inheritance:** Yes, Python supports it.
14. **Abstract Class:** A class that cannot be instantiated and contains abstract methods.
15. **Real-time Example:** A Car class with objects like BMW, Audi, and Tesla.